

Deploying on your Azure — one image, config-only

The Railway sandbox and your Azure production run the **same container image** built from this repo. Azure is turned on entirely by **environment variables** — no fork, no code branch. The adapters for Azure OpenAI, OIDC, an HTTP wallet gateway, Postgres, Redis and Application Insights are already in the codebase. Deployment is provisioning + wiring, not a rewrite.

- Same image
- Config-only switch
- Already Azure-aware

01 The principle

One image, promoted by config. Nothing is Railway-specific and nothing is Azure-specific in the code — the same build runs in both places, and each mode adapter is chosen by an env var at boot. Verify any deployment at `GET /api/health` → it returns `{ authMode, modelMode, walletMode, mockRailsAllowed, database, storeBackend, redisConfigured, oidcIssuer }`, so you can confirm prod is wired correctly without reading a log.

02 Azure resources to provision

RESOURCE	PURPOSE	NOTES
Azure Container Registry	Store the built image	<code>az acr build</code> from the repo <code>Dockerfile</code>
Azure Container Apps (or App Service for Containers)	Run the container	Min 1 replica; scale on HTTP. Health probe → <code>/api/health</code> . Also reads <code>WEBSITE_SITE_NAME</code> on App Service
Azure Database for PostgreSQL Flexible Server	Durable store (replaces the file-store)	Once <code>DATABASE_URL</code> is set, the <code>*_STORE_PATH</code> file fallbacks are ignored. Migrations run on boot
Azure OpenAI	The model	Create two deployments : a "large" (gpt-4o-class) and a "small/cheap" one — the tier map routes to them
Azure Cache for Redis (or Upstash)	Shared session store across replicas	Optional but recommended for > 1 replica
Azure Key Vault	Secrets — keys, tokens, connection strings	Reference into Container Apps as secret env vars
Application Insights	Telemetry / logs	App reads <code>APPLICATIONINSIGHTS_CONNECTION_STRING</code> natively
Identity provider Entra ID / partner OIDC	Consumer sign-in	Supplies the <code>MIAI_OIDC_*</code> values

03 The mock → prod switch, at a glance

CONCERN	SANDBOX	AZURE PROD	FLIPPED BY
Model	<code>mock</code> → <code>openai</code>	<code>azure</code>	<code>MIAI_MODEL_MODE</code> + <code>AZURE_OPENAI_*</code>
Identity	<code>mock</code>	<code>oidc</code>	<code>MIAI_AUTH_MODE</code> + <code>MIAI_OIDC_*</code>
Wallet	<code>mock</code>	<code>http</code>	<code>MIAI_WALLET_MODE</code> + <code>MIAI_WALLET_API_*</code>
Store	file on <code>/data</code>	Azure Postgres	presence of <code>DATABASE_URL</code>

04 Environment variables

Grouped by function. Set the "Prod value" column and leave the mock defaults behind; secrets go to Key Vault. Every var below is one the code actually reads.

4.1 · Core modes — the mock → real switches

VAR	PROD VALUE	PURPOSE
MIAI_AUTH_MODE	oidc	Real per-user identity (was mock)
MIAI_MODEL_MODE	azure or gateway	Route to Azure OpenAI (was mock)
MIAI_WALLET_MODE	http	Real billing gateway (was mock)
SANDBOX_MODE	unset	Turns off the sandbox turn-cap + banner
ALLOW MOCK RAILS I_UNDERSTAND MOCK RAILS IN PROD	leave unset	Safety: the app refuses to run mock rails in production unless <i>both</i> are set. Unset ⇒ forces real rails + strong secrets

4.2 · Data store

VAR	PROD VALUE	PURPOSE
DATABASE_URL	Azure Postgres conn string	Enables Postgres; disables the /data file store
PG_SSL_REJECT_UNAUTHORIZED	true	Azure Postgres requires SSL
UPSTASH_REDIS_REST_URL UPSTASH_REDIS_REST_TOKEN	if using Redis	Multi-replica session store

4.3 · Model — Azure OpenAI (MIAI_MODEL_MODE=azure)

VAR	PURPOSE
AZURE_OPENAI_ENDPOINT	https://<resource>.openai.azure.com
AZURE_OPENAI_API_KEY	Azure OpenAI key
AZURE_OPENAI_API_VERSION	e.g. 2024-08-01-preview
AZURE_OPENAI_DEPLOYMENT	small / cheap deployment name
AZURE_OPENAI_DEPLOYMENT_LARGE	large deployment name (the claude-sonnet/gpt-4o/opus tiers)
EMBEDDING_API_KEY · _BASE_URL · _MODEL	if using real semantic retrieval

Alternative — their model gateway (MIAI_MODEL_MODE=gateway): MIAI_MODEL_GATEWAY_URL, MIAI_MODEL_GATEWAY_KEY (or MIAI_MODEL_API_KEY), optional MIAI_MODEL_PASSTHROUGH=1.

4.4 · Identity / auth (MIAI_AUTH_MODE=oidc)

VAR	PURPOSE
MIAI_OIDC_ISSUER	Consumer OIDC issuer
MIAI_OIDC_CLIENT_ID · _CLIENT_SECRET	Consumer OIDC client
MIAI_OIDC_AUDIENCE · _JWKS_URL	Agents / API token validation

4.5 · Wallet / billing (http)

MIAI_WALLET_API_URL	Billing / wallet gateway base URL
MIAI_WALLET_API_KEY	Gateway key

4.6 · Channels

TELEGRAM_BOT_TOKEN · _SECRET · _USERNAME	Telegram (secret = webhook token + nonce signing; charset [A-Za-z0-9_-])
WHATSAPP_TOKEN · _PHONE_NUMBER_ID	WhatsApp channel (if enabled)

4.7 · App host & URLs

APP_BASE_URL · NEXT_PUBLIC_APP_URL	Public origin (deep links, webhooks)
MIAI_AGENTS_AUTH_URL · MIAI_CONSUMER_APP_URL	Cross-service URLs, if split (+ NEXT_PUBLIC_ twins)
CATALOG_DIR · CONSUMER_CATALOG_DIR	Baked into the image; override only if externalising the catalogue

4.8 · Telemetry & platform secrets — required once OIDC/prod, or boot fails

APPLICATIONINSIGHTS_CONNECTION_STRING	Azure App Insights
MIAI_SESSION_SECRET · OAUTH_TOKEN_SECRET · OAUTH_STATE_SECRET · EMBED_KEY_SECRET · WEBHOOK_SINK_SECRET · CRON_SECRET	Strong secrets — the hardening check fails boot otherwise
BRAVE_API_KEY / SEARCH_API_KEY · OPENWEATHER_API_KEY · YOUTUBE_API_KEY · PAYSTACK_* / STRIPE_SECRET_KEY	Optional — only if those tools/connectors are switched on

05 Deployment flow

☐ **Build & push** — `az acr build -r <acr> -t miai-web:<sha> .` (repo `Dockerfile` , health at `/api/health`)

☐ **Provision** the \$2 resources; create the two Azure OpenAI deployments; load all secrets into Key Vault

☐ **Create the Container App** with the image + \$4 env (secrets as Key Vault refs). Target port `3000` (or `PORT`)

☐ **Migrations** run automatically on first boot when `DATABASE_URL` is set — no separate step

☐ **Register webhooks** against the prod host: `setWebhook?`
Telegram `url=.../api/consumer/telegram/webhook&secret_token=$TELEGRAM_BOT_SECRET`

☐ **Verify** `GET /api/health` → expect `authMode:oidc · modelMode:azure · walletMode:http · database:postgres · mockRailsAllowed:false`

06 Multi-tenant & open decisions

WHITE-LABEL MODEL

Per-tenant, **not** per-fork: `workspaceId = brand = tenant`, and every consumer call is scoped to it — so each partner gets isolated memory, wallet and connectors off the one deployment. Registry: `apps/web/src/lib/tenant-brands.ts`. The sandbox is pinned to a single brand for a clean demo; the multi-tenant capability is intact.

DECISIONS STILL OPEN — CONFIG, NOT CODE

☐ **Model path:** Azure OpenAI direct vs their gateway

☐ **Identity:** their OIDC issuer vs Google

☐ **Whose Azure subscription** hosts it; tenant→partner map

☐ **Regions / residency:** Postgres + Azure OpenAI co-located per market

None of these require code changes — they select which env values go in \$4.